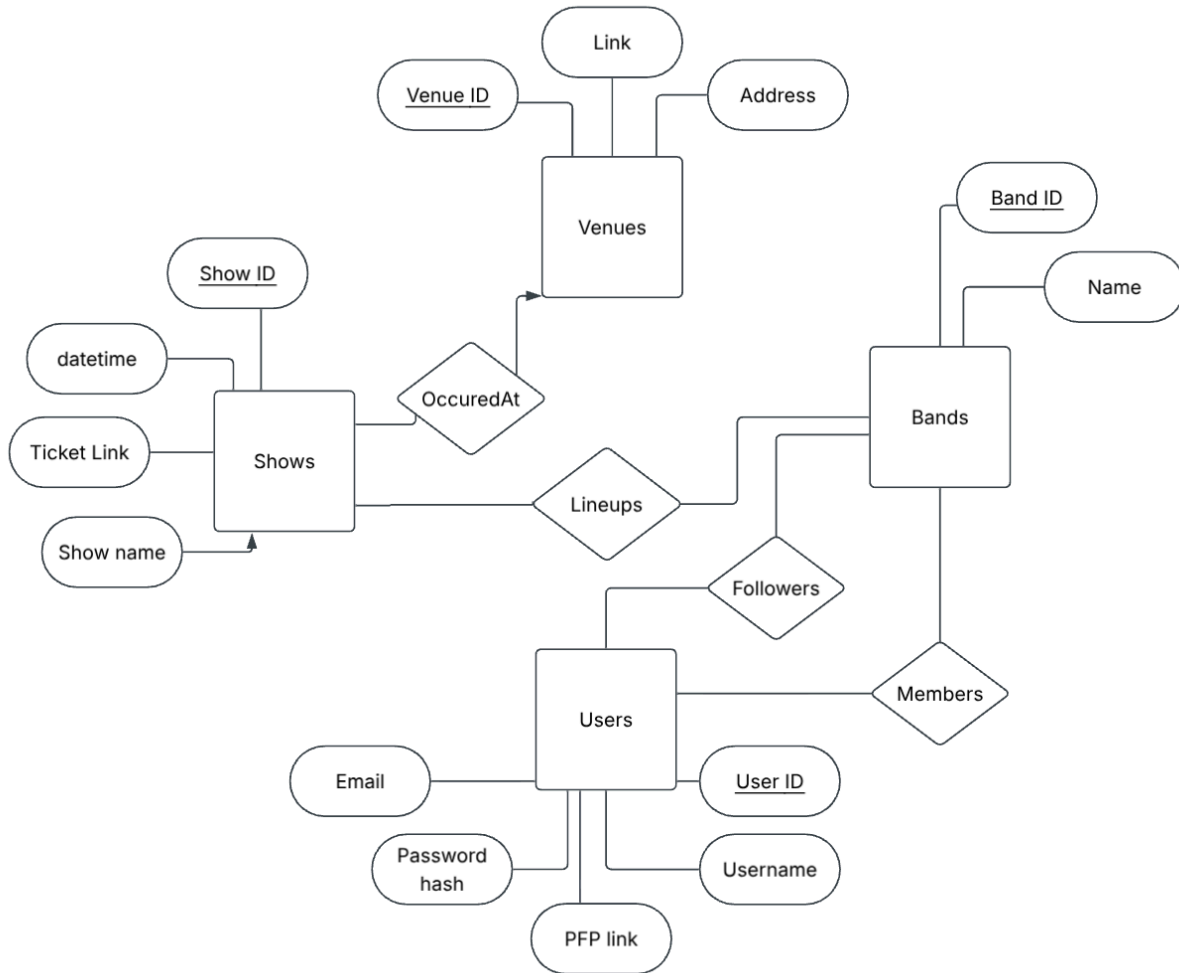


BCNF Decomposition - Proof



The above ERD highlights the basic design of our database. From it, I have extracted the functional dependencies that predominate our design. What follows is a proof of BCNF decomposition. First, I'll individually assess each equivalence class and show that it does not violate BCNF. Next, I'll decompose the entire schema by the recursive BCNF method we were taught.

To prove BCNF, I will show that every non-trivial FD has a superkey on the left hand side.

Functional Dependencies:

UserID -> username, PFPlink, passHash, email, role

ShowID -> datetime, ticketLink, showName, VenueID

VenueID -> Link, Address

BandID -> BandName

Decomposition - Full Schema

R(ShowID, ShowName, DateTime, TicketLink, VenueID, Address, VenueLink, BandID, BandName, UserID, Username, role, Email, PasswordHash, PFPLink)

Minimal Candidate Key - {ShowID, UserID, BandID}

Now, I'll decompose on the determinants of each dependency, those determinants being ShowID, UserID, VenueID, and BandID. The decomposition on the respective FDs results in the following:

Decompose by Shows FD - R violates BCNF

Shows(ShowID, datetime, ticketLink, showName, **VenueID**)

R1(ShowID, VenueID, Address, VenueLink, BandID, BandName, UserID, Username, role, Email, PasswordHash, PFPLink)

Decompose by Users FD - R1 violates BCNF

Shows(ShowID, datetime, ticketLink, showName, **VenueID**)

Users(UserID, Username, role, Email, PasswordHash, PFPLink)

R2(ShowID, VenueID, Address, VenueLink, BandID, BandName, UserID)

Decompose by Venues FD - R2 violates BCNF

Shows(ShowID, datetime, ticketLink, showName, **VenueID**)

Users(UserID, Username, role, Email, PasswordHash, PFPLink)

Venues(VenueID, link, address)

R3(ShowID, BandID, UserID, BandName)

Decompose by Bands FD - R3 Violates BCNF

Shows(ShowID, datetime, ticketLink, showName, **VenueID**)

Users(UserID, Username, role, Email, PasswordHash, PFPLink)

Venues(VenueID, link, address)

Bands(BandID, BandName)

R4(ShowID, BandID, UserID)

R4 is the minimal key of the relation, so it cannot be further reduced, it is in BCNF. The 4 equivalence classes highlighted above represent the basic framework of our database, and importantly they are all in BCNF. Consider the example of the Shows relation. As ShowID can determine all other attributes in the relation, as defined in the ShowID functional dependency, ShowID is a candidate key of that set. As ShowID is both a candidate key and the determinant for the only nontrivial FD within Shows, the Shows relation must be in BCNF. The same logic can be applied to the Users, Venues, and Bands table, which also satisfy BCNF.

Next, I'll evaluate the relationships in the database, which in our case are Members, Followers, and Lineups. First, I'll consider lineups.

Lineups is defined as **Lineups(band_id, show_id)**, where both band_id and show_id are foreign keys, and their combination serves as a composite primary key. Thus, there are no non-trivial functional dependencies to violate BCNF, and thus the relation is in BCNF. Next, we will evaluate the members relation.

Members is defined as **Members(user_id, band_id)**, where both user_id and band_id are foreign keys, and again their combination serves as a composite primary key. As there are no FD's relating user_id and band_id, the relation does not violate BCNF. Last, we'll evaluate the Followers relation.

Similarly, Followers is defined as **Followers(user_id, band_id)**, where both user_id and band_id are foreign keys. For the same reasons as described above, the Followers relation is also in BCNF.

One last note, the occurred_at relation between Shows and Venues manifests as Venue_id being used as a foreign key within the Shows relation. This does not violate BCNF, because the venue_id can be uniquely determined from show_id, which is also the candidate key for the relevant functional dependency.